HW10 Main — Technical Prototype

Exposure Triangle — click-through Flask prototype

Blake Vente (rv2459)

April 20, 2026

Table of Contents

Prototype
Video
Commit screenshot
Group — responsibilities
Low-fidelity prototype
What I did this week
Note on the reducer
Rubric checklist

Prototype

Continuation of the HW9 low-fidelity prototype, now implemented as a Flask application with an HTML/JS/jQuery/Bootstrap frontend. Learning arc + quiz arc are reachable end-to-end; the click-through walks every screen the rubric requires (home → 3 lessons → 5 quizzes → result).

- **Repo.** https://github.com/rvente/exposure-triangle
- **Run locally.** `uv sync && uv run playwright install chromium && uv run flask --app app run --port 5000` — opens on `localhost:5000`.
- **Main entry.** `/` (home + Start button).

Video

~1 min click-through: https://youtu.be/17tHtb9QTxc

Commit screenshot

GitHub view of the HW10 Flask-prototype commit on https://github.com/rvente/exposure-triangle:

rvente / exposure-triangle

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security and quality | Insights | Settings

Commit eb502bf

Browse files

rvente committed 11 minutes ago

```
HW10 main: Flask technical prototype

- 4 routes: /, /learn/<n>, /quiz/<n>, /result, plus /api/{state,event,reset}
- Pure reducer + SQLite state store (per-session persistence of answers,
  attempts, latency, entry timestamps)
- Content externalized to data/lessons.json and data/quizzes.json
- Reducer unit tests (8 passing) + Playwright click-through e2e
- hw10_main.md writeup, hw10_warmup.pdf carried forward
```

main

1 parent ca81d7d commit eb502bf

Filter files...

- .gitignore
- Makefile
- README.md
- app.py
- data
  - lessons.json
  - quizzes.json
  - hw10_main.md
  - hw10_warmup.pdf
  - pyproject.toml
  - reducer.py
  - state_store.py
- static
  - css
    - app.css
  - js
    - app.js
    - backend.js
  - renders
- templates
  - base.html
  - home.html
  - learn.html
  - quiz.html
  - result.html
- tests
  - conftest.py
  - test_e2e.py
  - test_reducer.py
- uv.lock

Some content is hidden

Large Commits have some content hidden by default. Use the searchbox below for content that may be hidden.

47 files changed +1411 -2 lines changed

Search within code

- .gitignore +8
- Makefile +17
- README.md +27 -2
- app.py +157
- data/lessons.json +41

```
@@ -0,0 +1,41 @@
+ {
+   "1": {
+     "id": "iris",
+     "title": "Camera Iris",
+     "technical_term": "aperture / f-stop",
+     "body_md": "**Your eyes already do this.** Squinting makes distant things a little sharper — but also darker. The Camera Iris works the same way: wide open lets in more light but blurs the background; very closed keeps everything in focus but needs more light.\n\nThe side effect of a wide-open Camera Iris can be beautiful: background lights turn into soft glowing circles called *bokeh*.",
+     "images": [
+       {"src": "renders/bokeh_f1.2.avif", "label": "Wide open — shallow focus, strong bokeh"},
+       {"src": "renders/bokeh_f5.6.avif", "label": "Moderate — balanced"},
+       {"src": "renders/bokeh_f22.0.avif", "label": "Very closed — deep focus"}
+     ],
+     "metaphor_line": "The sensor is a bucket — the Camera Iris is how wide the mouth opens.",
+     "next": "/learn/2"
+   },
+   "2": {
+     "id": "duration",
+     "title": "Capture Duration",
+     "technical_term": "shutter speed",
+     "body_md": "The Camera Iris controls how *wide* the light stream is. The **Capture Duration** controls how *long* the sensor collects light.\n\nSports photographers use a short Capture Duration to freeze action. Night photographers use a long one to gather light trails.\n\nA long Capture Duration makes moving subjects **streak** across the frame — we call this *motion streaking* in this lesson.",
```

```
+    "images": [
+      {"src": "renders/motion_s0.0.avif", "label": "Short — motion frozen"},
+      {"src": "renders/motion_s1.2.avif", "label": "Medium — some streaking"},
+      {"src": "renders/motion_s3.5.avif", "label": "Long — heavy motion streaking"}
```

Commit eb502bf on rvente/exposure-triangle — HW10 main: Flask technical prototype (47 files changed)

Group — responsibilities

Solo CVN group of 1 — every role below was filled by me (Blake Vente, rv2459).

Part	Role	Person
1. Learning	Architecting the data (`data/lessons.json`)	Blake Vente
1. Learning	Implementing the UI (`templates/home.html`, `templates/learn.html`)	Blake Vente
1. Learning	Testing the click-through (`tests/test_e2e.py` — learning leg)	Blake Vente
2. Quiz	Architecting the data (`data/quizzes.json`)	Blake Vente
2. Quiz	Implementing the UI (`templates/quiz.html`, `templates/result.html`)	Blake Vente
2. Quiz	Testing the click-through (`tests/test_e2e.py` — quiz leg)	Blake Vente
—	State + scoring logic (`reducer.py`, `state_store.py`)	Blake Vente

Low-fidelity prototype

The final low-fi prototype was submitted with the HW10 warm-up (`writeup/hw10_warmup.pdf`; PowerPoint source is `writeup/exposure_triangle.pptx`). The HW10 technical prototype covers the core teaching-and-quiz flow shown in the lo-fi.

Not yet implemented (next up, HW11): the lo-fi also sketches an adaptive- difficulty path where wrong quiz answers route the learner to an easier variant and right answers unlock harder variants. For HW10 the quiz uses a fixed 5-question sequence with a second-chance mechanic (first wrong grays out the option; second wrong reveals the correct answer). The adaptive branching is the primary HW11 iteration.

What I did this week

- **Scaffolded the Flask app.** `app.py` exposes `/`, `/learn/<n>`, `/quiz/<n>`, `/result`, plus `POST` endpoints that persist user choices on every page.
- **Separated concerns.** `reducer.py` is a pure function `reduce(state, event, now_ms, *, answer_key)`; `state_store.py` persists sessions keyed by cookie in SQLite (`instance/state.db`).
- **Ported content to JSON.** `data/lessons.json` (3 lessons) and `data/quizzes.json` (5 quizzes with second-chance mechanic). No content is hard-coded in HTML — each route renders from the JSON.
- **Tests.** Reducer unit tests cover state transitions and scoring edge cases. Playwright e2e test walks the full click-through under `xvfb-run`.

Note on the reducer

Writing the core as a `reduce(state, event)` pure function looks like overkill for an app this small, but the call paid for itself quickly:

1. **Speed.** The reducer was fast to write — once the state shape was named, each event became a 3–5 line case. The bulk of the work was content (lessons + quizzes JSON), not control flow.
2. **Testability under branching paths.** Adaptive difficulty (HW11) turns the quiz into a branching tree rather than a fixed sequence. A pure reducer means I can unit-test every branch in isolation without spinning up Flask, Playwright, or a DB — just feed it events and assert on the returned state.
3. **Two-backend symmetry → local-first deployment.** The `Backend` interface (`static/js/backend.js`) has a `FlaskBackend` and a stub `LocalBackend`. Because the reducer is pure JS/Python-portable logic, the `LocalBackend` can run the same transitions entirely in the browser with `localStorage` — no server required. That lets the site be statically served (e.g. Cloudflare Pages) and run local-first, which matters for learner

accessibility: no account, no network, no tracking. The reducer is the shared kernel that makes both paths real.

Rubric checklist

- [x] Flask backend, HTML/JS/jQuery/Bootstrap frontend.
- [x] Home screen with Start button.
- [x] Backend stores user choices on every page (quiz answers + lesson entry timestamps, via `state_store.py`).
- [x] Content in JSON (`data/lessons.json`, `data/quizzes.json`), rendered into templates — not hard-coded.
- [x] 4 routes: `/`, `/learn/<n>`, `/quiz/<n>`, `/result`.
- [x] Each page shows data, gives instructions, records data, and advances.
- [x] Quiz result page shows score reflecting correct/incorrect answers.
- [x] Single-user assumption (session cookie keyed).